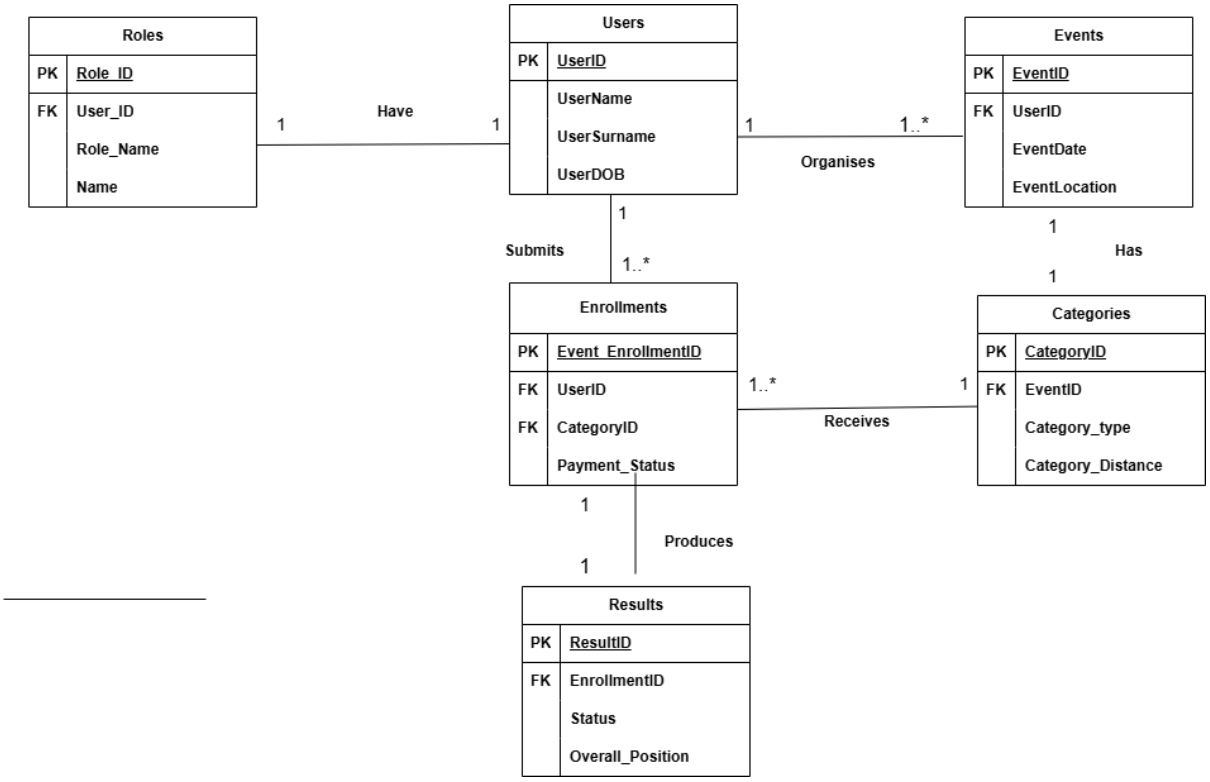
NTWANANO BALOYI
ST10483015
PROGRAMMING 1B
04 SEPTEMBER 2026

# PROGRAMMING 1B

## ASSIGNMENT 1

# SECTION A: ENTITY RELATION DIAGRAM



# SECTION B: API ENDPOINT PLAN

## 1.AUTHENTICATION

SECTION B: API ENDPOINT PLAN

Authentication

| HTTP Method | ROUTE | DESCRIPTION | ROLE REQUIRED | REQUEST BODY | EXPECCTED RESPONSE |
|---|---|---|---|---|---|
| POST | /api/auth/register | Register a new account as an Organiser or Participant | Public | {“name", "email", "password", "role”} | 201 Created — {user: {id, name, email, role}, token} |
| POST | /api/auth/login | Authenticate a user and issue an access token | Public | {“email", "password”} | 200 OK — {user: {...}, token} |
| POST | /api/auth/refresh-token | /api/auth/refresh-token | Authenticated | {“refresh Token”} | 200 OK — {token} |
| POST | /api/auth/logout | Invalidate the current session/token | Authenticated | none | 200 OK — {message: "Logged out”} |

## USER PROFILE

| **HTTP Method** | **Route** | **Description** | **Role Required** | **Request Body** | **Expected Response** |
|---|---|---|---|---|---|
| GET | /api/users/me | Retrieve the logged-in user's own profile | Authenticated | none | 200 OK — user object |
| PUT | /api/users/me | Update the logged-in user's profile details | Authenticated | {“name", "phone", ...} | 200 OK — updated user object |
| DELETE | /api/users/me | Deactivate / delete the logged-in user's own account | Authenticated | none | 200 OK — {message} |
| GET | /api/users/me/results | Retrieve the logged-in participant's personal performance history | Participant | none | 200 OK — array of result objects |

# EVENTS

| **HTTP Method** | ROUTE | DESCRIPTION | ROLE REQUIRED | REQUEST BODY | EXPECTED RESPONSE |
|---|---|---|---|---|---|
| GET | /api/events | Browse/search all upcoming events (filter by date, location, type) | Public | none *(query params: location=&type=& date=)* | 200 OK — array of event objects |
| GET | /api/events/:id | Get full details of a single event, including its categories and route | Public | none | 200 OK — event object |
| POST | /api/events | Create a new event | Organiser | {“name", "description", "event date", "location", "event type”} | 201 Created — event object |
| PUT | /api/events/:id | Update an existing event (owner only) | Organiser | {fields to update} | 200 OK — updated event object |
| DELETE | /api/events/:id | Cancel / delete an event (owner only) | Organiser | none | 200 OK — {message} |
| GET | /api/events/:id/weather | Get live weather forecast for the event's location/date | Authenticated | none | 200 OK — {temperature, conditions, windspeed, forecast Date} |
| GET | /api/events/:id/routes | Get route/map information for an event | Public | none | 200 OK — array of route objects0 |

## CATEGORIES

| HTTP METHOD | ROUTE | DESCRIPTION | ROLE REQUIRED | REQUEST BODY | EXPECTED RESPONSE |
|---|---|---|---|---|---|
| GET | /api/events/:eventId/categories | List all categories (e.g. 5km, 10km, 21km) for an event | Public | none | 200 OK — array of category objects |
| POST | /api/events/:eventId/categories | Add a new category to an event (owner only) | Organiser | {“name", "distance", "entry fee", "max_participants”} | 201 Created — category object |
| PUT | /api/categories/:id | Update a category's details (owner only) | Organiser | {fields to update} | 200 OK — updated category object |
| DELETE | /api/categories/:id | Remove a category (owner only) | Organiser | None | 200 OK — { message } |

## ENROLLMENTS

| HTTP METHOD | ROUTE | DESCRIPTION | ROLE REQUIRED | REQUEST BODY | EXPECTED RESPONSE |
|---|---|---|---|---|---|
| POST | /api/categories/:categoryId/enrolments | Enter the logged-in participant into a category (generates a bib number) | Participant | {} *(user id taken from auth token)* | 201 Created — enrolment object |
| GET | /api/users/me/enrolments | List the logged-in participant's own entries | Participant | none | 200 OK — array of enrolment objects |
| GET | /api/events/:eventId/enrolments | List all entrants for an event (owner only) | Organiser | none | 200 OK — array of enrolment objects |
| DELETE | /api/enrolments/:id | Withdraw / cancel an entry (owner participant only) | Participant | none | 200 OK — {message} |

## RESULTS

| HTTP METHOD | ROUTE | DESCRIPTION | ROLE REQUIRED | REQUEST BODY | EXPECTED RESPONSE |
|---|---|---|---|---|---|
| POST | /api/enrolments/:enrolmentId/results | Capture a finish-line result for an entrant | Organiser | {“finish time", "gun_time", "overall_position", "category_position", "status”} | 201 Created — result object |
| GET | /api/events/:eventId/results | Get the full results list / leaderboard for an event | Public | none | 200 OK — array of result objects |
| PUT | /api/results/:id | Correct or update a captured result (owner organiser only) | Organiser | {fields to update} | 200 OK — updated result object |
| DELETE | /api/results/:id | Remove an incorrectly captured result | Organiser | none | 200 OK — {message} |

## SECTION 3: SQL SQRIPT

1. ROLE TABLE

SQLQuery1.s...udent (60))*

```
CREATE DATABASE RACEDAY;
USE RACEDAY;

CREATE TABLE ROLES (ROLE_ID INT IDENTITY(1001,1) PRIMARY KEY,
                    ROLE_NAME VARCHAR(255) NOT NULL,
                    NAME VARCHAR(255) NOT NULL);


   INSERT INTO ROLES(ROLE_NAME,NAME)
   VALUES('ORGANISER','THABO'),
         ('PARTICIPANT','EMILY'),
         ('ORGANISER','AISHA'),
         ('PARTICIPANT','LIAM'),
         ('ORGANISER','NALEDI');

          SELECT*FROM ROLES
```

99 % No issues found

Results Messages

| | ROLE_ID | ROLE_NAME | NAME |
|---|---|---|---|
| 1 | 1001 | ORGANISER | THABO |
| 2 | 1002 | PARTICIPANT | EMILY |
| 3 | 1003 | ORGANISER | AISHA |
| 4 | 1004 | PARTICIPANT | LIAM |
| 5 | 1005 | ORGANISER | NALEDI |

## 2. USERS TABLE

SQLQuery1.s...udent (60))*

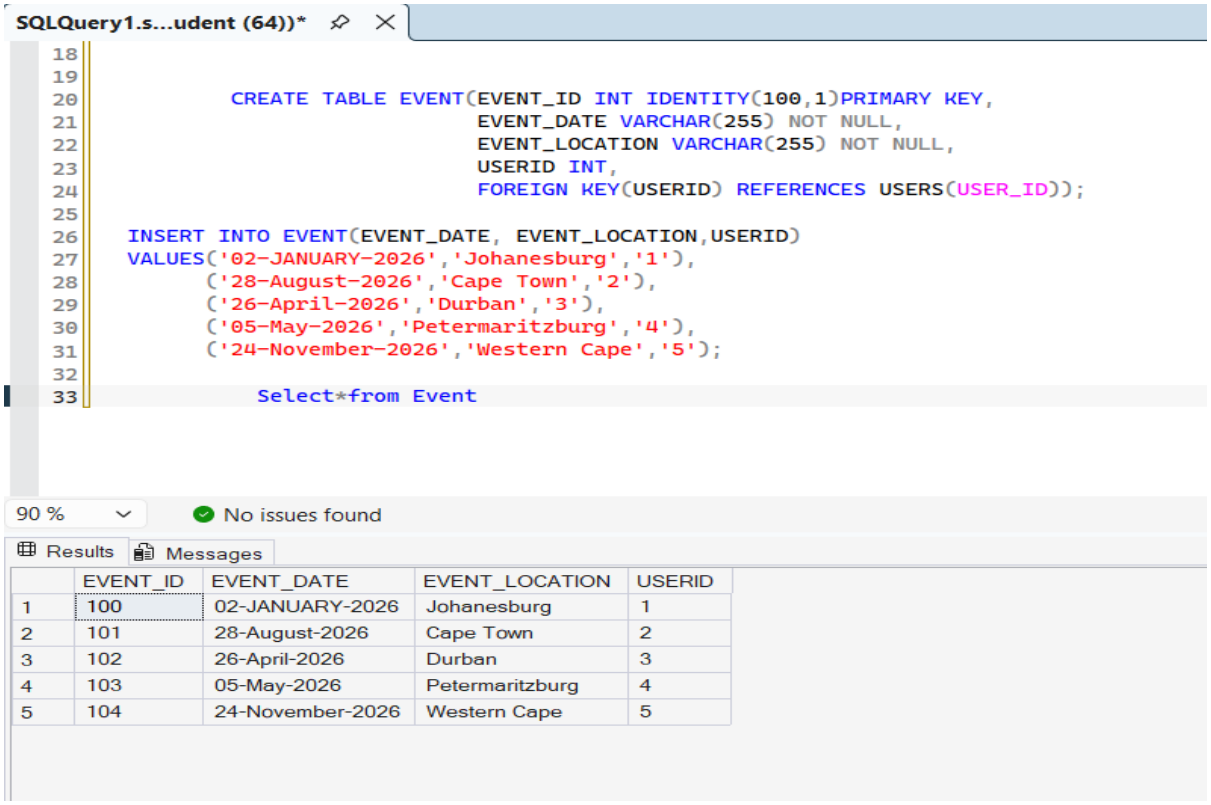
```
CREATE TABLE USERS(USER_ID INT IDENTITY(100,1) PRIMARY KEY,
        USER_NAME VARCHAR(255) NOT NULL,
        USER_SURNAME VARCHAR(255) NOT NULL,
        USER_DOB VARCHAR(255) NOT NULL,
        ROLEID INT,
        FOREIGN KEY(ROLEID) REFERENCES ROLES(ROLE_ID));

   INSERT INTO USERS(USER_NAME, USER_SURNAME,USER_DOB,ROLEID)
   VALUES ('THABO','MOKOENA','25-May-2006',1001),
          ('EMILY','CARTER','26-September-2012',1002),
          ('AISHA','PATEL','26-April-2003',1003),
          ('LIAM','VAN WYK','04-October-2000',1004),
          ('NALEDI','KHUMALO','11-June-2008',1005);

          SELECT*FROM USERS
```

99 % No issues found

Results Messages

| | USER_ID | USER_NAME | USER_SURNAME | USER_DOB | ROLEID |
|---|---|---|---|---|---|
| 1 | 100 | THABO | MOKOENA | 25-May-2006 | 1001 |
| 2 | 101 | EMILY | CARTER | 26-September-2012 | 1002 |
| 3 | 102 | AISHA | PATEL | 26-April-2003 | 1003 |
| 4 | 103 | LIAM | VAN WYK | 04-October-2000 | 1004 |
| 5 | 104 | NALEDI | KHUMALO | 11-June-2008 | 1005 |

## 3.EVENTS TABLE



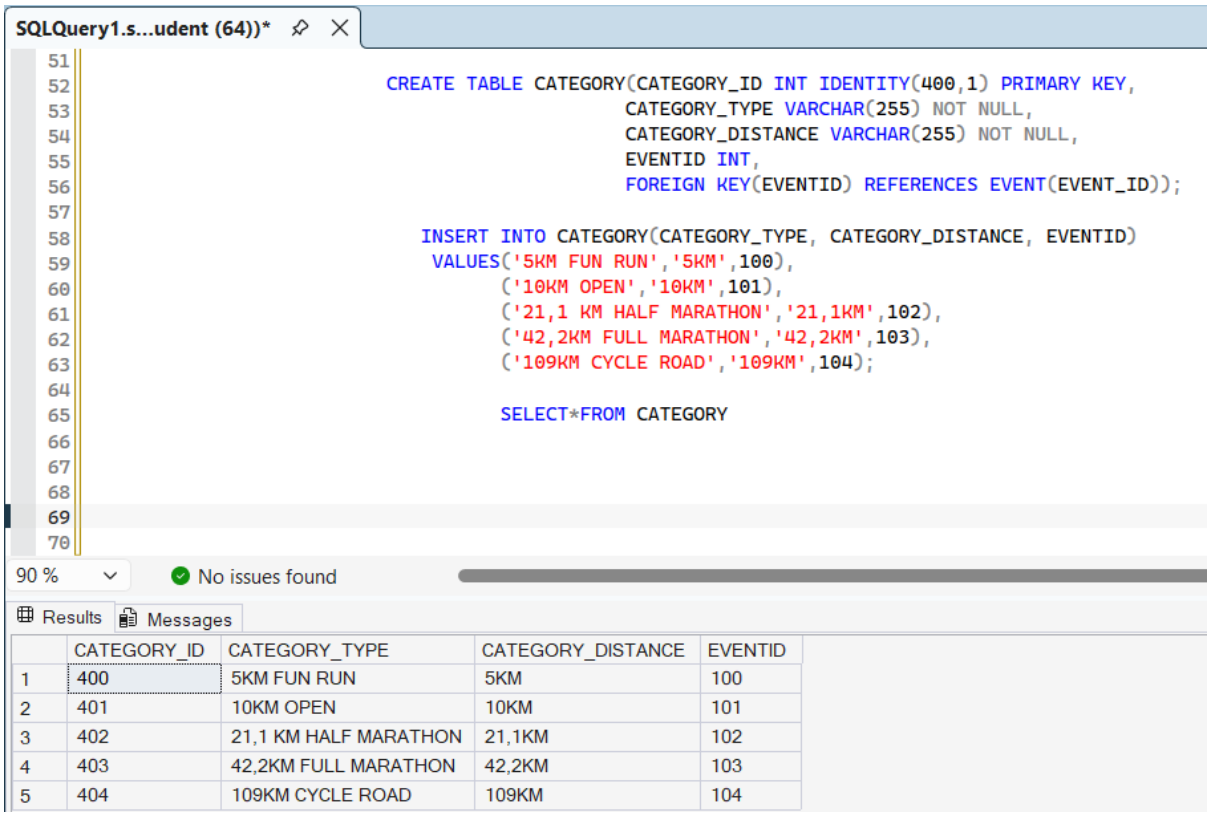
```
SQLQuery1.s...udent (64))*

18
19
20          CREATE TABLE EVENT(EVENT_ID INT IDENTITY(100,1)PRIMARY KEY,
21                             EVENT_DATE VARCHAR(255) NOT NULL,
22                             EVENT_LOCATION VARCHAR(255) NOT NULL,
23                             USERID INT,
24                             FOREIGN KEY(USERID) REFERENCES USERS(USER_ID));
25
26  INSERT INTO EVENT(EVENT_DATE, EVENT_LOCATION,USERID)
27  VALUES('02-JANUARY-2026','Johanesburg','1'),
28        ('28-August-2026','Cape Town','2'),
29        ('26-April-2026','Durban','3'),
30        ('05-May-2026','Petermaritzburg','4'),
31        ('24-November-2026','Western Cape','5');
32
33            Select*from Event
```

90 % No issues found

Results Messages

| | EVENT_ID | EVENT_DATE | EVENT_LOCATION | USERID |
|---|---|---|---|---|
| 1 | 100 | 02-JANUARY-2026 | Johanesburg | 1 |
| 2 | 101 | 28-August-2026 | Cape Town | 2 |
| 3 | 102 | 26-April-2026 | Durban | 3 |
| 4 | 103 | 05-May-2026 | Petermaritzburg | 4 |
| 5 | 104 | 24-November-2026 | Western Cape | 5 |

## 4. CATEGORIES



```
SQLQuery1.s...udent (64))*

51
52                  CREATE TABLE CATEGORY(CATEGORY_ID INT IDENTITY(400,1) PRIMARY KEY,
53                                        CATEGORY_TYPE VARCHAR(255) NOT NULL,
54                                        CATEGORY_DISTANCE VARCHAR(255) NOT NULL,
55                                        EVENTID INT,
56                                        FOREIGN KEY(EVENTID) REFERENCES EVENT(EVENT_ID));
57
58                INSERT INTO CATEGORY(CATEGORY_TYPE, CATEGORY_DISTANCE, EVENTID)
59                 VALUES('5KM FUN RUN','5KM',100),
60                       ('10KM OPEN','10KM',101),
61                       ('21,1 KM HALF MARATHON','21,1KM',102),
62                       ('42,2KM FULL MARATHON','42,2KM',103),
63                       ('109KM CYCLE ROAD','109KM',104);
64
65                       SELECT*FROM CATEGORY
66
67
68
69
70
```

90 % No issues found

Results Messages

| | CATEGORY_ID | CATEGORY_TYPE | CATEGORY_DISTANCE | EVENTID |
|---|---|---|---|---|
| 1 | 400 | 5KM FUN RUN | 5KM | 100 |
| 2 | 401 | 10KM OPEN | 10KM | 101 |
| 3 | 402 | 21,1 KM HALF MARATHON | 21,1KM | 102 |
| 4 | 403 | 42,2KM FULL MARATHON | 42,2KM | 103 |
| 5 | 404 | 109KM CYCLE ROAD | 109KM | 104 |

## 5. ENROLLMENTS

SQLQuery1.s...udent (64))*

```
66
67   CREATE TABLE ENROLLMENT(ENROLLMENT_ID INT IDENTITY(500,1) PRIMARY KEY,
68                           PAYMENT_STATUS VARCHAR(255) NOT NULL,
69                           USERID INT,
70                           CATEGORYID INT,
71                           FOREIGN KEY(USERID)REFERENCES USERS(USER_ID),
72                           FOREIGN KEY(CATEGORYID) REFERENCES CATEGORY(CATEGORY_ID));
73
74                       INSERT INTO ENROLLMENT(PAYMENT_STATUS, USERID,CATEGORYID)
75                       VALUES('PAYMENT OUTSTANDING',1,400),
76                             ('PAYMENT RECEIVED',2,401),
77                             ('PENDING',3,402),
78                             ('PAYMENT FAILED',4,403),
79                             ('REFUNDED',5,404);
80
81                             SELECT*FROM ENROLLMENT
82
83
84
85
```

90 % No issues found

Results Messages

| | ENROLLMENT_ID | PAYMENT_STATUS | USERID | CATEGORYID |
|---|---|---|---|---|
| 1 | 500 | PAYMENT OUTSTANDING | 1 | 400 |
| 2 | 501 | PAYMENT RECEIVED | 2 | 401 |
| 3 | 502 | PENDING | 3 | 402 |
| 4 | 503 | PAYMENT FAILED | 4 | 403 |
| 5 | 504 | REFUNDED | 5 | 404 |

## 6.RESULTS

SQLQuery1.s...udent (64))*

```
82
83
84   CREATE TABLE RESULTS(RESULT_ID INT IDENTITY(600,1)PRIMARY KEY,
85                        STATUS VARCHAR(255)NOT NULL,
86                        OVERALL_POSITION VARCHAR(255) NOT NULL,
87                        ENROLLMENTID INT,
88                        FOREIGN KEY(ENROLLMENTID) REFERENCES ENROLLMENT(ENROLLMENT_ID));
89
90                INSERT INTO RESULTS(STATUS, OVERALL_POSITION,ENROLLMENTID)
91                VALUES('finished','1st position',500),
92                   ('finished','2ND POSITION',501),
93                   ('did not start','3RD POSITION',502),
94                   ('disqualified','4TH POSITION',503),
95                   ('did not start','5TH POSITION',504);
96                   select*from results
```

99 % No issues found

Results Messages

| | RESULT_ID | STATUS | OVERALL_POSITION | ENROLLMENTID |
|---|---|---|---|---|
| 1 | 600 | finished | 1st position | 500 |
| 2 | 601 | finished | 2ND POSITION | 501 |
| 3 | 602 | did not start | 3RD POSITION | 502 |
| 4 | 603 | disqualified | 4TH POSITION | 503 |
| 5 | 604 | did not start | 5TH POSITION | 504 |

## REFERENCES AND LINKS